INKLOOM AI

MASTER IMPLEMENTATION PROMPT

Only the detailed instructions required to complete the existing project, implement all remaining Round 1 features, and add secure social-media publishing for human-approved content.

COPY-READY BUILD INSTRUCTION

Use this prompt with the existing project ZIP and the two reference PDFs. It instructs the coding assistant to inspect, modify, test and complete the existing application—not to create a disconnected replacement.

ROLE AND OBJECTIVE

Act as a senior full-stack architect, product engineer, AI engineer, security engineer and QA lead. You are working on the existing INKLOOM AI project supplied as a ZIP, together with the Round 1 proposal PDF and the existing master specification PDF.

Your task is to CONTINUE AND COMPLETE THE EXISTING PROJECT. Do not create an unrelated replacement application and do not merely produce a plan. First inspect the entire repository, identify the current implementation, preserve working functionality and visual identity, then implement every remaining requirement described below. The Round 1 proposal is the source of truth for the original product scope. The new requirement from the product owner is to add connected social-media publishing for content that has passed human approval.

Deliver working, integrated code—not placeholder buttons, static screens, mock API responses, local-only demo state or documentation that claims an unfinished feature is complete. Where a provider or external credential is unavailable, implement the full integration architecture, configuration, clear setup instructions and honest disabled/unconnected state. Never pretend a real external action succeeded when it did not.

1. START WITH THE EXISTING ZIP — AUDIT BEFORE EDITING

1. Inspect every folder, source file, route, component, service, database schema, configuration file, test and documentation file in the supplied ZIP.

2. Map each existing screen and feature to its actual implementation. Distinguish UI-only screens from features connected to persistent backend logic.

3. Reuse and improve the current React/Vite frontend, existing Express/TypeScript server, current service structure and existing design wherever practical. Do not discard working code or rewrite the whole application without a clear technical reason.

4. Identify all missing, incomplete, mocked, duplicated, insecure or disconnected functionality. Create an implementation checklist and track each item to completion.

5. Check the current API contract against actual server routes. Implement missing routes and update the contract to match the real implementation.

6. Check the database schema against actual runtime persistence. A schema file alone is not database integration.

7. Remove production dependence on localStorage for shared business data. Local storage may be used for harmless UI preferences or temporary drafts, but submissions, approvals, comments, versions, users, brand data, analytics and publishing records must be persisted in the backend database.

8. Do not break existing navigation, responsive layouts, role-based views or demo workflows while upgrading the application.

2. PRODUCT SCOPE — COMPLETE ALL ROUND 1 FEATURES

Implement the following complete product capabilities. Each must have a working frontend, backend/API, database persistence, permissions, validation, loading/empty/error states and tests where applicable.

A. Brand Knowledge Hub

• Provide a central workspace for each brand with brand name, description, audience, positioning, values, voice attributes, preferred style, prohibited style, language and channel guidance.

• Support create, view, edit, archive and permission-controlled management of brand profiles.

• Allow approved brand facts, product details, approved claims, terminology, phrases, assets and reference material to be stored and retrieved.

• Associate every piece of brand knowledge with its owner, status, timestamps and source/reference where relevant.

• Make brand knowledge available to the AI analysis pipeline as structured context; do not rely on hard-coded sample data.

B. Brand Guideline Manager

• Create, edit, organize, publish and archive guideline sections.

• Support tone, voice, vocabulary, grammar, formatting, capitalization, punctuation, hashtag, emoji, CTA, visual identity and channel-specific rules.

• Each rule must include a clear description, severity, examples where useful, active status and applicable brand/channel.

• Maintain guideline history so changes can be traced.

• Ensure the analyzer uses the currently active approved guideline version.

C. Content Submission Workspace

• Allow authorized users to submit text, captions, campaign copy, documents and supported creative assets.

• Capture title, content type, brand, channel, campaign, author and submission metadata.

• Support draft, submitted, analysis-in-progress, changes-requested, resubmitted, approved, rejected and published states.

• Validate required fields and file types/size. Store uploaded assets securely using configured object storage; do not expose private storage credentials.

• Show submission detail, current status, assigned reviewer and activity timeline.

D. AI Brand Voice and Content Analysis

• Analyze content against the selected brand profile and active guidelines.

• Return an overall score and clear category scores, findings, severity, explanation, evidence excerpt, violated rule and actionable recommendation.

• Identify on-brand strengths as well as issues.

• Support an analysis result with a stable ID, model/provider metadata, prompt/rule version and timestamp for auditability.

• Make analysis reproducible enough to understand which brand knowledge and guideline version were used.

• Do not fabricate findings or claim certainty when evidence is weak. Show uncertainty or request human review where appropriate.

E. Terminology Consistency Checker

• Compare content against approved terms, preferred spellings, prohibited terms, product names, capitalization and phrase rules.

• Highlight exact text locations and show preferred replacements with explanations.

• Support brand-specific and channel-specific terminology.

• Allow authorized brand managers to maintain the terminology list.

F. Approved Fact Verification

• Compare factual claims with the brand's approved facts and product information.

• Show the claim, matching approved source/fact, match status and confidence.

• Distinguish verified, conflicting, unsupported and needs-human-verification claims.

• Never invent a source or mark an unsupported statement as verified.

• Allow reviewers to open the source/reference used for a fact.

G. Unsupported Claim Detection

• Identify claims that are not supported by the available approved brand knowledge.

• Explain why a claim needs evidence and suggest safe wording or a request for verification.

• Do not treat absence of a fact as proof that a claim is false.

• Provide a clear path for a reviewer or brand manager to add/approve a source fact.

H. Visual Brand Consistency Review

• Support creative asset upload and preview.

• Provide a practical MVP visual review: allow a reviewer to compare an uploaded creative with stored brand assets, palette guidance and visual rules.

• Where image-analysis capability is configured, return evidence-based observations and identify the relevant asset/rule. If no vision provider is configured, show a transparent manual-review workflow instead of fake AI results.

• Do not claim pixel-perfect logo detection, exact layout measurement or advanced visual intelligence unless it is genuinely implemented and tested.

I. Accessibility Review

• Implement text-level accessibility checks that can be reliably evaluated, such as missing or unclear link text where applicable, excessive all-caps, readability warnings, descriptive alt-text presence for uploaded images and caption/transcript reminders for relevant media.

• Provide actionable recommendations and explain the scope of each check.

• Do not claim full WCAG conformance from text-only checks. Clearly distinguish automated checks from manual review.

J. AI Correction Assistant

• Provide suggested rewrites for individual findings and for the full content.

• Preserve the author's intent, factual meaning, required terms, brand voice and channel constraints.

• Show original and suggested versions side by side, with highlighted changes where feasible.

• Allow the user to accept, reject or edit each suggestion.

• Never silently overwrite the original submission.

• Any accepted change must create a new immutable content version and trigger re-analysis before it can be approved.

K. Review and Approval Workflow

• Implement assignment, reviewer queue, review detail, comments, request-changes, approve and reject actions.

• Enforce valid state transitions on the server, not only in frontend code.

• Require a human reviewer with the correct permission to approve.

• Record actor, action, timestamp, previous state, new state and optional reason.

• Prevent authors from approving their own submission if the configured governance policy disallows it.

• A changed or newly generated version must not inherit approval from an older version.

L. Comments and Collaboration

• Support persistent comments and threaded replies linked to a submission and, where possible, a specific content version or text range.

• Include author, timestamp, edit/delete policy and read/unread state where appropriate.

• Support reviewer feedback and author replies.

• Store all comments in the backend and show them consistently across devices.

• Restrict comment visibility and moderation actions by role.

M. Version Control and Revision History

• Store immutable snapshots of every submitted, edited, corrected, approved and published content version.

• Show version number, author, date, change summary and approval status.

• Support side-by-side comparison or a clear change view.

• Preserve the exact version that was approved and the exact version later published.

• Never mutate an approved snapshot in place.

N. Governance Analytics Dashboard

• Calculate metrics from real persisted records, not hard-coded demo numbers.

• Include submission volume, approval/rejection/change-request counts, average review time, brand compliance scores, common violation categories, workflow status distribution and activity over time.

• Provide filters by date range, brand, channel, user and status where relevant.

• Ensure metrics respect user permissions and tenant/brand access.

• Show empty states when no data exists and explain metric definitions.

O. Role-Based Access Control and User Management

• Implement secure authentication and server-enforced authorization.

• Support roles such as Admin, Brand Manager, Contributor/Creator and Reviewer/Approver, with permissions configurable to the product's governance model.

• Provide user listing, invitation or registration flow as appropriate, role assignment, activation/deactivation and profile management.

• Enforce least privilege on every protected API route and every sensitive operation.

• Do not rely on a frontend role flag or localStorage value as proof of authorization.

3. NEW REQUIRED FEATURE — CONNECT SOCIAL ACCOUNTS AND PUBLISH APPROVED CONTENT

Build a complete social-media connection and approved-publishing module inside INKLOOM AI. This is a required addition to the current implementation.

A. Social account connection centre

• Add a Social Accounts page in settings or the brand workspace.

• Let an authorized user connect and disconnect supported social accounts through each provider's official OAuth authorization flow.

• Start with provider integrations that can be implemented using currently available official APIs and permissions, such as Meta Facebook Pages and eligible Instagram Professional accounts, and LinkedIn organization/member publishing where the account and app have the required access.

• Treat X or any additional network as an extensible adapter only when the relevant official API access and permissions are available. Do not promise unsupported capabilities.

• Show provider, account/page name, account type, connection owner, granted scopes, connected/disconnected/expired/error state and last successful verification.

• Explain missing provider permissions and provide setup guidance.

• Never ask users to enter their social-media passwords into INKLOOM AI. Never scrape websites or automate a logged-in browser as a substitute for an official API.

B. OAuth and token security

• Perform OAuth authorization-code exchange on the backend.

• Validate state and redirect URI; use PKCE when supported/required.

• Store access tokens and refresh tokens encrypted at rest using a server-side encryption key or managed secret service.

• Never send provider tokens, client secrets or encryption keys to the browser, localStorage, logs or analytics.

• Implement token refresh, expiry detection, revocation handling and reconnect prompts.

• Request only the minimum provider scopes needed for the chosen publishing features.

• Provide secure disconnect and token revocation/deletion behavior.

C. Approved content to social channel workflow

• From an approved submission, allow an authorized user to select one or more connected social accounts.

• Prepare a channel-specific post preview with text/caption, hashtags, link, image/video attachments, alt text where supported and provider-specific limits.

• Support channel variants because the same campaign may require different copy for different networks.

• Each channel-specific variant must be stored as its own immutable version.

• If a caption, asset or other publishable content is changed after analysis or approval, mark that variant as requiring re-analysis and fresh human approval.

• Never publish an unapproved version. Never assume approval of the source submission automatically approves a materially changed channel variant.

• Before publishing, verify that the exact version selected is approved, the account is connected, the provider token is valid and required assets are available.

• Display the exact final content and target account to the user before scheduling or publishing.

D. Publishing actions

• Support publish now and schedule for later.

• Provide timezone-aware scheduling and clear display of the scheduled local time and timezone.

• Use a background job/worker for scheduled publishing; do not depend on a browser tab remaining open.

• Track queued, scheduled, publishing, published, failed, cancelled and needs-reconnect states.

• Use idempotency keys or equivalent safeguards to prevent duplicate posts during retries.

• Handle provider rate limits, transient failures, invalid media, expired permissions and provider validation errors.

• Retry only safe transient failures with bounded backoff. Do not retry permanent validation or permission errors indefinitely.

• Save provider post ID, canonical URL when available, published timestamp, target account, exact content version, actor and response metadata needed for support/audit.

• Do not show “Published” until the provider confirms success. If a provider response is uncertain, show an explicit unknown/verification-needed state and reconcile before retrying.

E. Social publishing UI

• Add Social Accounts connection management.

• Add a channel selector and channel-specific preview to the approved submission detail.

• Add a publish/schedule confirmation screen.

• Add a publishing calendar or scheduled-post list.

• Add publishing history with filters and provider response/error details.

• Add reconnect and failed-post recovery actions.

• Show which user initiated, scheduled, cancelled or retried an action.

• Make clear that INKLOOM AI is a brand-governance and approval platform with publishing capability, not an unmoderated auto-poster.

F. Social publishing data model

Create suitable relational tables or equivalent normalized entities, including:

• social_accounts: provider, external account ID, display name, account type, encrypted token references, granted scopes, expiry, connection status, owner/brand and timestamps.

• social_post_variants: source submission, immutable content version, target provider/account, caption/body, media references, channel metadata, approval state and timestamps.

• publish_jobs: variant, account, scheduled time/timezone, job state, retry count, idempotency key, last error and timestamps.

• publish_results: job, provider post ID, provider URL, published time, response metadata and final status.

• provider_webhook_events or equivalent event log where provider webhooks are used.

Add indexes, foreign keys, uniqueness constraints and tenant/brand access controls.

G. Social publishing API

Implement and document protected backend endpoints equivalent to:

• GET /api/social/providers

• GET /api/social/accounts

• POST /api/social/connect/:provider/start

• GET /api/social/connect/:provider/callback

• DELETE /api/social/accounts/:id

• POST /api/submissions/:id/social-variants

• GET /api/submissions/:id/social-variants

• POST /api/social-variants/:id/analyse

• POST /api/social-variants/:id/submit-for-approval

• POST /api/social-variants/:id/approve

• POST /api/social/publish

• POST /api/social/schedule

• GET /api/social/publishing

• GET /api/social/publishing/:id

• POST /api/social/publishing/:id/cancel

• POST /api/social/publishing/:id/retry

These are functional requirements; adapt route naming to the existing project conventions if needed. Every endpoint must validate input, authenticate the user, authorize the brand/account access, and return consistent error responses.

4. BACKEND, DATABASE AND API COMPLETION

• Use the existing backend stack where practical and make it the authoritative source of business data.

• Connect the application to a real PostgreSQL database (or the already selected managed PostgreSQL/Supabase service if configured).

• Add proper migrations, constraints, indexes, seed data for development and environment-based configuration.

• Implement complete CRUD and workflow APIs for users, brands, guidelines, facts, terms, assets, submissions, analyses, findings, comments, versions, approvals, audit events, analytics and social publishing.

• Ensure all reads and writes are scoped to the authenticated user's permitted brands/tenants.

• Use transactions for operations that must succeed or fail together, especially approval/version creation and publish-job creation.

• Use pagination, filtering, sorting, validation and consistent error handling.

• Update API documentation from the actual implementation. Do not document endpoints that do not exist.

• Add health/readiness checks that verify required dependencies without exposing secrets.

• Remove silent local fallback behavior in production. If the database is unavailable, return a clear service error rather than pretending data was saved.

5. AI PROVIDER AND ANALYSIS PIPELINE SECURITY

• Move all AI provider calls and secret API keys to the server. Do not store a Gemini/OpenAI/other provider key in localStorage or frontend environment variables exposed to the browser.

• Add a provider abstraction so the application can switch models without rewriting business logic.

• Keep prompts, model settings and provider keys in server-side configuration/secrets.

• Validate and sanitize user input and uploaded content before sending it to a model.

• Treat user-submitted content and retrieved documents as untrusted data, never as system instructions.

• Require structured model output validated against a schema before saving it.

• Store the model/provider, prompt version, guideline version, brand context version and timestamp for every analysis.

• Add rate limits, usage limits, timeout handling, retry rules and clear AI service failure states.

• Do not expose chain-of-thought or hidden reasoning. Provide concise evidence-based explanations and recommendations.

• If AI is unavailable, allow manual review and show that AI analysis could not be completed; do not fabricate a score.

6. AUTHENTICATION, WORKFLOW AND AUDIT RULES

• Implement secure login/session or JWT handling with appropriate expiry, refresh/revocation strategy and password hashing if passwords are managed by this application.

• Protect all privileged routes server-side.

• Implement a central workflow state machine and reject invalid state transitions.

• Require an explicit authorized human action for final approval.

• Store audit events for login/security events where appropriate, user/role changes, brand/guideline changes, submission actions, analysis, corrections, approvals, rejections, comments, social-account connections, publish scheduling, cancellation, retries and provider outcomes.

• Audit events should be append-only for normal application users.

• Store timestamps consistently in UTC and render in the user's configured timezone.

• Make operations safe against duplicate requests and race conditions.

7. FRONTEND AND EXPERIENCE REQUIREMENTS

Preserve the existing INKLOOM AI visual language and working layouts while improving consistency and usability. Ensure the product is responsive and usable on desktop, tablet and mobile.

Review and complete the existing pages/components where present, including dashboard, brand hub, guidelines, submission workspace, analysis/results, correction workspace, review queue, approval detail, history/version history, analytics, users, audit log, settings and admin areas.

Add any missing screens needed for the full workflow, including:

• Social Accounts and provider connection states.

• Social channel variant editor and preview.

• Publish/schedule confirmation.

• Publishing calendar/list and publishing history.

• Failed publishing details and reconnect flow.

For every page, implement loading, empty, success, validation, permission-denied and error states. Avoid dead buttons, fake charts, hard-coded counters and nonfunctional controls. Ensure keyboard accessibility, visible focus states, semantic labels, readable contrast and clear status text. Do not rely on color alone to communicate approval or failure.

8. SECURITY, PRIVACY AND RELIABILITY

• Validate every API input on the server and encode output safely.

• Use secure HTTP headers, CORS allowlists, CSRF protection where cookie-based sessions are used, rate limiting and secure cookie settings.

• Protect against SQL injection, XSS, broken access control, IDOR, SSRF and unsafe file uploads.

• Restrict file type/size, scan files where feasible, use private object storage and short-lived signed URLs for private assets.

• Keep secrets in deployment secret storage and provide a safe .env.example containing placeholders only.

• Never commit API keys, tokens, passwords, private URLs or production credentials.

• Encrypt social tokens at rest and redact sensitive provider data from logs.

• Add structured server logging, request IDs and useful operational error messages.

• Apply retention/deletion rules for user data and connected account tokens.

• Do not claim legal, privacy or platform-policy compliance without verifying the implementation and provider requirements.

9. TESTING AND ACCEPTANCE CRITERIA

Write and run tests for the core workflows. At minimum verify:

• A user cannot access a brand or submission outside their permissions.

• A contributor can create and submit content but cannot perform an unauthorized approval.

• A reviewer can request changes, approve or reject only when permitted.

• Every edit/correction creates a new immutable version.

• Approval is attached to the exact version and is invalidated for a changed version.

• AI findings cite the relevant guideline/fact/rule and unsupported claims are not marked verified.

• Comments, versions, approvals and analytics persist after refresh and across devices.

• Dashboard metrics are computed from database records.

• Social OAuth state validation and callback handling work securely.

• Social tokens never appear in browser storage, API responses or logs.

• A disconnected/expired account cannot publish.

• An unapproved or changed version cannot be published.

• A confirmed publish stores the provider post ID and URL where available.

• A failed publish is accurately marked failed and can be safely retried without duplicate posting.

• Scheduled jobs execute without an open browser tab.

• API errors, empty states, loading states and permission errors are handled clearly.

Provide unit tests, API/integration tests and end-to-end tests for the main user journey. Include a test report showing what passed, failed or remains blocked by missing external credentials/provider approval.

10. DELIVERY REQUIREMENTS — DO THE WORK, NOT JUST DESCRIBE IT

Complete the implementation in logical phases while keeping the application runnable:

Phase 1: repository audit, implementation map, environment/configuration cleanup and database connection/migrations.

Phase 2: authentication, roles, backend authorization and persistent core entities.

Phase 3: complete Round 1 brand knowledge, guidelines, submissions, analysis, terminology, facts, unsupported claims and accessibility/visual review capabilities.

Phase 4: correction assistant, immutable versions, comments, review/approval workflow and audit trail.

Phase 5: real analytics and user management.

Phase 6: social OAuth connections, channel variants, re-analysis/re-approval, scheduling worker, publishing, history and failure recovery.

Phase 7: security hardening, automated tests, responsive QA, deployment and documentation.

After each phase, run the application and relevant tests. Fix regressions before proceeding. Do not stop after generating a checklist or scaffolding.

Final deliverables:

1. Updated complete source code in the existing project structure.

2. Database migrations and safe development seed data.

3. Updated API contract and architecture notes.

4. Updated .env.example with every required variable and no real secrets.

5. Setup instructions for local development and production deployment.

6. Social provider setup guide explaining app registration, redirect URLs, required permissions/scopes, limitations and credential configuration.

7. Test suite and honest test results.

8. A feature completion matrix marking each requirement as implemented, partially implemented or blocked, with evidence such as route/component/test names.

9. A concise demo script showing the complete flow: create brand → configure guidelines → submit content → analyze → correct → review → approve → connect social account → create channel variant → re-analyse/re-approve → schedule/publish → inspect publishing history and audit log.

Do not mark a feature complete merely because a page exists. A feature is complete only when the user can perform the action, the backend enforces the rules, the data persists, failures are handled and the workflow is tested. If external provider access prevents a live end-to-end test, clearly identify that dependency and provide a mock-free manual verification procedure for when credentials are available.

11. NON-NEGOTIABLE PRODUCT RULES

1. Preserve the existing project and improve it; do not replace it without necessity.

2. Round 1 requirements remain in scope; the new social publishing capability is added on top.

3. The backend database is the source of truth for shared product data.

4. Security and authorization must be enforced on the server.

5. AI must use approved brand knowledge and active guidelines; it must not invent evidence.

6. Human approval is mandatory before publication.

7. Publish only the exact immutable version that was approved for the target channel/account.

8. Any content or asset change after approval requires fresh analysis and approval.

9. Use official social platform APIs and OAuth only; never collect social passwords or scrape.

10. Never expose AI keys, OAuth secrets or social tokens in the frontend.

11. Never fake an AI result, approval, database save or successful post.

12. Keep the UI honest about unavailable integrations, permissions and incomplete checks.

13. Test the complete workflow and provide evidence of what is actually working.

FINAL INSTRUCTION TO THE IMPLEMENTING AI

Begin by inspecting the supplied ZIP and both reference PDFs. Then implement the remaining work in the existing repository according to this prompt. Use the Round 1 PDF to preserve the original product requirements and terminology. Treat this prompt as the updated implementation instruction, including the newly requested social account connection and approved publishing feature.

Do not respond with only recommendations. Make the code changes, run the application and tests, fix issues, and report the exact files changed, commands run, working features, remaining external dependencies and deployment steps. If something cannot be implemented because credentials, provider access or infrastructure are missing, implement the safe integration boundary and explain the precise blocker without pretending it is complete.